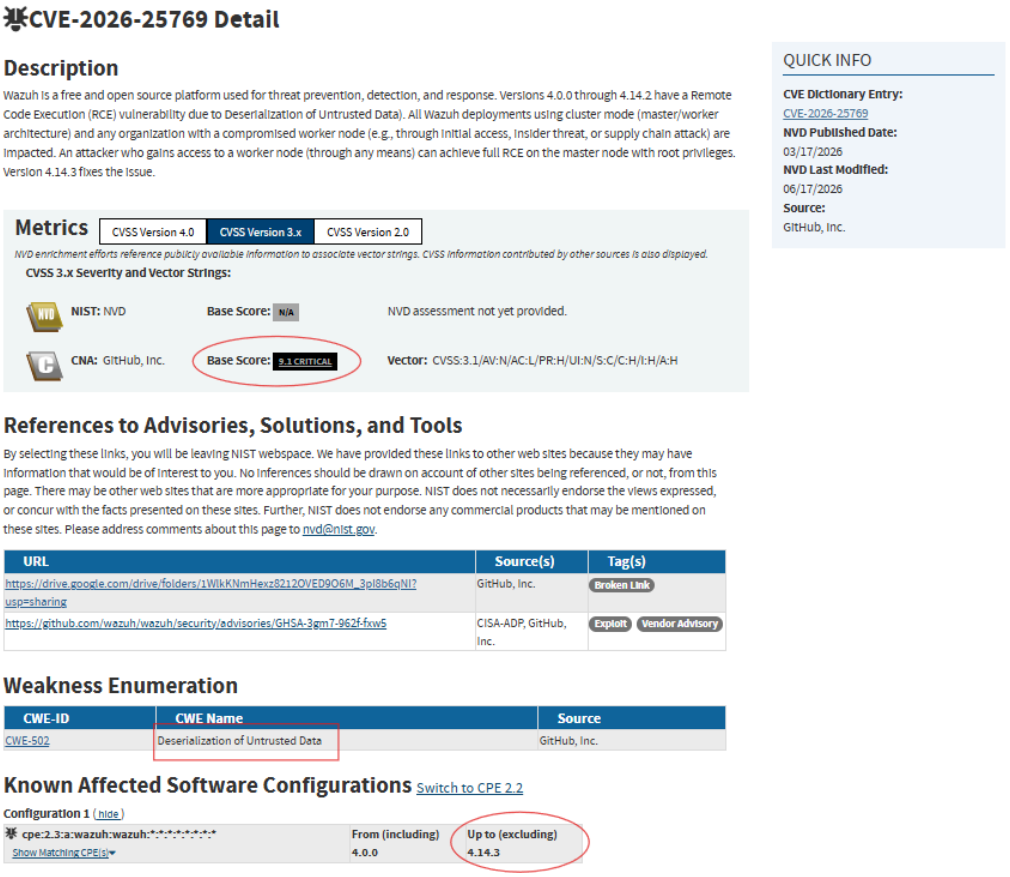
Wazuh: CVE-2026-25769



CVE-2026-25769 Detail

Description

Wazuh is a free and open source platform used for threat prevention, detection, and response. Versions 4.0.0 through 4.14.2 have a Remote Code Execution (RCE) vulnerability due to Deserialization of Untrusted Data). All Wazuh deployments using cluster mode (master/worker architecture) and any organization with a compromised worker node (e.g., through initial access, insider threat, or supply chain attack) are impacted. An attacker who gains access to a worker node (through any means) can achieve full RCE on the master node with root privileges. Version 4.14.3 fixes the issue.

QUICK INFO

CVE Dictionary Entry: CVE-2026-25769
NVD Published Date: 03/17/2026
NVD Last Modified: 06/17/2026
Source: GitHub, Inc.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD — **Base Score:** N/A — NVD assessment not yet provided.

CNA: GitHub, Inc. — **Base Score:** 9.1 CRITICAL — **Vector:** CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

URL	Source(s)	Tag(s)
https://drive.google.com/drive/folders/1WlkKNmHexz8212OVED9O6M_3pI8b6qNI?usp=sharing	GitHub, Inc.	Broken Link
https://github.com/wazuh/wazuh/security/advisories/GHSA-3gm7-962f-fxw5	CISA-ADP, GitHub, Inc.	Exploit, Vendor Advisory

Weakness Enumeration

CWE-ID	CWE Name	Source
CWE-502	Deserialization of Untrusted Data	GitHub, Inc.

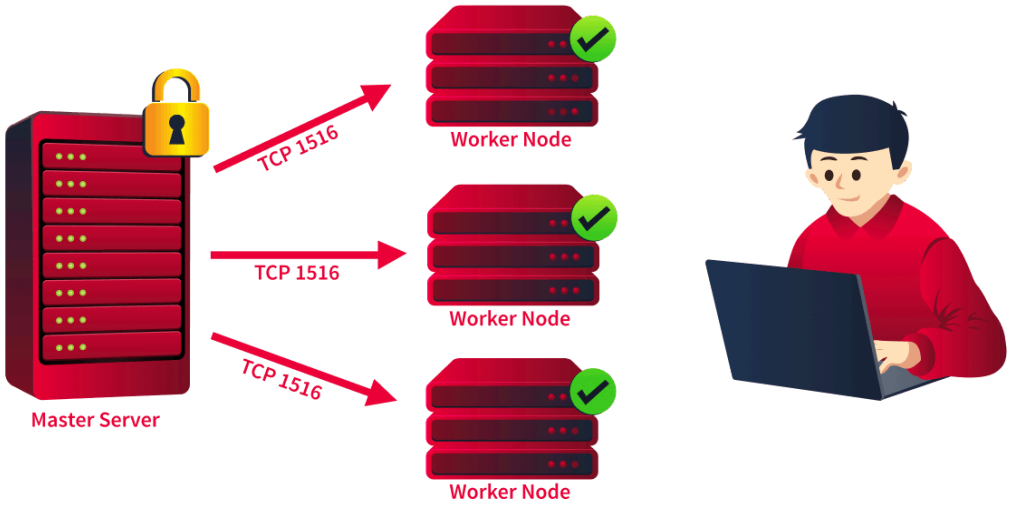
Known Affected Software Configurations

Switch to CPE 2.2

Configuration 1 (hide)

cpe:2.3:a:wazuh:wazuh:*:*:*:*:*:*:*:*	From (including)	Up to (excluding)
Show Matching CPE(s)	4.0.0	4.14.3

```
Severity: Critical
CVSS Score: 9.1
CVE-ID: CVE-2026-25769
CWE: CWE-502 (Deserialization of Untrusted Data)
Affected Versions: 4.0.0 through 4.14.2
Patched Version: 4.14.3
```



A master node coordinates one or more worker nodes to distribute workloads across large environments. Every alert, every log, every detection rule flows through the master node. Now imagine an attacker gains access to a single worker node and, from there, executes arbitrary commands on the master with root privileges. That is exactly what CVE-2026-25769 allows.

Workers connect to the master over TCP port `1516`. All communication between nodes is encrypted using a shared Fernet key defined in the cluster configuration file (`ossec.conf`). This encryption ensures that only nodes with the correct key can exchange messages. However, once a node is authenticated, the master implicitly trusts all content it receives, and this is where the problem lies.

Wazuh workers send requests to the master through DAPI, where the requests are converted to JSON, sent, and then converted back into Python objects using `json.loads()` and `object_hook`.

The vulnerability is located in `framework/wazuh/core/cluster/common.py` in the `as_wazuh_object()` function (lines 1830-1866). This function is registered as the `object_hook` parameter in `json.loads()`, which means every JSON object deserialised from cluster messages passes through it for custom transformation.

```
# framework/wazuh/core/cluster/common.py:1839-1848

def as_wazuh_object(dct: Dict):
    try:
        if '__callable__' in dct:
            encoded_callable = dct['__callable__']
            funcname = encoded_callable['__name__']
            if '__wazuh__' in encoded_callable:
```

```
                wazuh = Wazuh()
                return getattr(wazuh, funcname)
            else:
                qualname = encoded_callable['__qualname_
_'].split('.')
                classname = qualname[0] if len(qualname) >
1 else None
                module_path = encoded_callable['__module_
_']   # NO VALIDATION
                module = import_module(module_path)
# ARBITRARY IMPORT
                if classname is None:
                    return getattr(module, funcname)
# RETURNS ARBITRARY FUNCTION
                else:
                    return getattr(getattr(module, classnam
e), funcname)
```

This function **deserializes JSON data and can import and return arbitrary Python functions specified by the input**, which is dangerous if an attacker can control the JSON.

The deserialised function is executed in `framework/wazuh/core/cluster/dapi/dapi.py`.

```
# Line 705: Deserialisation with vulnerable hook
request = json.loads(request, object_hook=c_common.as_wazuh
_object)

# Line 248: Execution of deserialized function
data = f(**f_kwargs)  # f = subprocess.getoutput, f_kwargs
= {"cmd": "COMMAND"}
```

Attack Chain

1. The attacker compromises a worker node (via initial access, insider threat, or supply chain attack)

2. The worker already has the shared Fernet key for cluster communication
3. The attacker sends a malicious DAPI request via `LocalClient.execute()`
4. The message is encrypted with the cluster key and sent over TCP to the master on port `1516`
5. The master's `APIRequestQueue.run()` receives and deserialises the message
6. `as_wazuh_object()` processes the `__callable__` key, imports `subprocess`, and returns `subprocess.getoutput`
7. `DistributedAPI.run_local()` calls `subprocess.getoutput(cmd="...")` with root privileges
8. The attacker's command executes on the master node

Problematic reasons

1. The master trusts all authenticated worker messages without validating the content.
2. The `as_wazuh_object()` function has no module allowlist restricting which Python modules can be imported during deserialisation.

Detection

The exploit is difficult to detect because the malicious request uses the same encrypted Wazuh cluster communication channel and port **1516** as legitimate DAPI traffic, so the network traffic itself may look normal; instead, defenders should look for unusual request volumes, unexpected connections to port 1516, and especially suspicious child processes spawned by `wazuh-clusterd`, unexpected files or reverse-shell connections from the master, persistence mechanisms, and suspicious imports or errors in `cluster.log`; the main mitigation is to **upgrade Wazuh to 4.14.3 or later**, which adds an allowlist that prevents arbitrary module imports during deserialization, while additional protection includes restricting port 1516 to trusted workers, monitoring worker integrity, segmenting the cluster network, rotating cluster keys after suspected compromise, using least privilege, and enabling auditing and file-integrity monitoring.

References

- https://tryhackme.com/room/wazuhcve202625769
- https://cryptography.io/en/latest/fernet/
- https://nvd.nist.gov/vuln/detail/CVE-2026-25769